Class Wingspan (Extends JFrame)

Overview: This class is the runner class of the program and contains methods required for transitioning between and displaying panels.

Attributes

- JPanel panel: The current panel being displayed on the frame

Methods

- void main(String[] args): Program runner function; first displays the MenuPanel to the player
- void backToMenu(): Returns the player to the menu after the game is over
- void setPanel(JPanel panel): Set the attribute panel to the provided panel and update which panel is being displayed
- void startGame(Wingspan frame, int playerCount): Creates a new Game and runs setPanel to update the panel to a new GamePanel

Class Game

Overview: This class is the bulk of the program and contains attributes and methods required for initialization, scoring, and turn mechanics. It also contains global variables required throughout the entire game.

Attributes

- HashSet<BirdCard> birdDeck: A set of all the BirdCards in the game, one created for each instance of the enum BirdCard
- HashSet<BonusCard> bonusDeck: A set of all the BonusCards in the game, one created for each instance of the enum BonusCard
- Wingspan frame: The instance that contains necessary methods for transition of panels and the beginning and ending of the game
- GamePanel panel: The GamePanel used to communicate with the graphical user interface
- int[] playerPoints: An array containing each player's points
- Player[] players: An array containing all Player objects used to play the game, one for each player
- int playerTurn: The index of the Player instance corresponding to the player whose turn it is
- int round: The current round being played

Methods

- Game(Wingspan frame, int playerCount): Initializes the object to create the players array with playerCount amount of Players and to hold the parent class in the frame attribute
- BirdCard drawBird(): Removes and returns a BirdCard from the HashSet birdDeck
- BonusCard drawBonus(): Removes and returns a BonusCard from the HashSet bonusDeck
- void endGame(): Calculates scores and displays the leaderboard/victors; ends the game
- Player getPlayer(int i): Returns the Player at index i of the array players
- void handOutCards(): Draws five BirdCards per player allows the player to select a total of five items out of five foodTokens and the five cards, then allows the player to select one of two drawn BonusCards; discards unused cards
- void nextTurn(): Sets the playerTurn to the next index and runs all necessary functions for the player's turn
- void nextRound(): Sets up the next round by decrementing each Player's starting actionCubes and adding up points based on the criteria of the current round

Class MenuPanel (Extends JPanel)

Overview: This class is the graphical user interface for the startup menu and communicates primarily the number of players to the Wingspan class.

Attributes

- Wingspan frame: The Wingspan instance containing methods for panel transition and game startup

Methods

- MenuPanel(Wingspan frame): Initializes the object to hold the parent class in the frame attribute
- void mouseClicked(MouseEvent e): Communicates the information of the graphical user interface to the Wingspan class, primarily the number of players and when to start the round
- void mousePressed(MouseEvent e): Necessary for the MouseListener interface; unused
- void mouseReleased(MouseEvent e): Necessary for the MouseListener interface; unused
- void mouseEntered(MouseEvent e): Necessary for the MouseListener interface; unused
- void mouseExited(MouseEvent e): Necessary for the MouseListener interface; unused
- void paint(): Updates graphics in response to the user's clicks

Class GamePanel (Extends JPanel)

Overview: This class is the graphical user interface for the main game and comunicates actions launched by the player. It will display the board of each player and will give the option to display other players' boards during each player's turn.

Attributes

- Game game: The Game instance that the graphical user interface communicates with
- Player playerDisplaying: The Board of the Player being displayed

Methods

- GamePanel(Wingspan frame): Initializes the object to hold the parent class in the frame attribute and to set the playerDisplaying attribute
- void mouseClicked(MouseEvent e): Communicates the information of the graphical user interface to the Game class including the action being done and the board being displayed
- void mousePressed(MouseEvent e): Necessary for the MouseListener interface; unused
- void mouseReleased(MouseEvent e): Necessary for the MouseListener interface; unused
- void mouseEntered(MouseEvent e): Necessary for the MouseListener interface; unused
- void mouseExited(MouseEvent e): Necessary for the MouseListener interface; unused
- void paint(): Updates graphics in response to the user's clicks

Class Player

Overview: This class will contain attributes related to each player's items and board as well as functions required for playing cards and executing actions.

Attributes

- int actionCubes: The remaining number of actions the corresponding player can take in a given round
- Board board: The Board class corresponding to this Player instance
- ArrayList<BirdCard> birdCards: The hand of BirdCards corresponding to the player
- ArrayList<BonusCard> bonusCards: The hand of BonusCards corresponding to the player
- int[] foodTokens: An array of size 4 holding the corresponding player's number of food tokens of each category
- Game game: The Game instance that the Player communicates with for drawing cards (establishes a bidirectional relationship)
- boolean marker: Whether or not the corresponding player holds the first player marker

Methods

- Player(Game game): Initializes the Player object with a bidirectional relationship to the Game instance
- int bonusAction(BonusCard bonus): Returns the bonus points given by the bonus
- boolean enoughFor(BirdCard bird): Determines whether or not the player has enough food tokens and eggs to play the given BirdCard
- Board getBoard(): Returns the Board instance for Game and GamePanel

- void placeAction(int habitat): Executes the action for the given habitat before decrementing the actionCube's column to execute preceeding actions
- boolean playCard(BirdCard, int habitat): Plays the given BirdCard in the given habitat if requirements are met; returns whether or not the play was legal and successful
- void runAction(int id): Runs the action corresponding to the given actionId
- void setActionCubes(int a): Sets the actionCubes by the Game class

Class Board

Overview: This class will be used primarily for storing a grid of the cards played by each player as well as food points and eggs stored on the board. It will contain necessary functions for modifying such attributes as well as for moving and launching actions.

Attributes

- int[] actionCube: The x- and y- values of the current actionCube on the Board
- int[] finishedActions: The y-value of finished actionCubes on the board for the purpose of displaying
- Spot[][] grid: The matrix of Spots that hold BirdCards, eggs, and food tokens
- Player player: The Player instance corresponding to the Board

Methods

- Board(Player player): Initializes the Board with a bilateral relationship to the Player
- void act(): Runs the action corresponding to the Spot on the coordinate of the actionCube
- int getEggCount(): Returns the sum of the eggs on all the Spots in the grid
- void moveAction(): Moves the actionCube to the left and continues to act until the actionCube is finished, where it sets the coordinate values of actionCube to -1 and increments the corresponding value in finishedActions
- void placeAction(int habitat): Decrements the Player's actionCubes and sets the coordinate of the Board's actionCube
- void playCard(BirdCard card, int habitat): Places the BirdCard in the corresponding Spot in the grid and gives the option to run the Action onPlay
- boolean placeEgg(int gridX, int gridY): Places the egg on the Spot corresponding to the given coordinates; returns whether or not the placement was successful

Class Spot

Overview: This class will be representative of each spot on the player's board. It will contain attributes of each spot - such as the card holding the spot, food points, and eggs - as well as functions for modifying attributes (i.e. placing a card, adding and removing eggs and food points).

Attributes

- BirdCard card: The BirdCard occupying the Spot, if any
- int eggCount: The number of eggs on the Spot, if occupied
- int[] foodPoints: The number of each food token cached as a point on the Spot, if occupied
- ArrayList<BirdCard> tuckedCards: The BirdCards tucked underneath the occupying BirdCard, if occupied

Methods

- void assignCard(): Assigns a BirdCard to occupy the Spot if unoccupied
- boolean addEgg(): Adds an egg to the Spot if possible; returns whether or not the placement was successful
- void addFood(int[] food): Adds a food point to the Spot
- boolean occupied(): Returns whether or not the Spot has a BirdCard occupying it
- void removeEgg(): Decrements the eggCount of the Spot

Class BirdFeeder

Overview: This class is simply meant to represent the bird feeder and the management of the dice.

Attributes

- ArrayList<Integer> dice: The five dice the BirdFeeder contains

Methods

- void roll(): Randomly assigns five values from 1-6 to the dice
- boolean allSame(): Returns whether or not all the values are the same in order to prompt the user to reroll

Enum BirdCard

Overview: This enum structures the bird cards with necessary values for the initialization of all of them in one long file. It contains constants that will vary for each card but should otherwise not be modified.

Attributes

- final String name: The name of the BirdCard
- final int actionId: The ID of the action that the BirdCard holds
- final boolean onPlay: Whether the action is upon the BirdCard’s placement or upon activation

- final int eggMax: The number of eggs that can be placed on the Spot the BirdCard occupies
- final int habitats: A bitmask of the habitats the BirdCard can be placed in
- final int nest: The nest type of the BirdCard
- final double wingspan: The wingspan of the BirdCard
- int[] foodCost: The number of food tokens of each variety required to play the BirdCard

Methods

- BirdCard(String name, int maxEgg, int habitats, double wingspan, int[] foodCost, int actionId, boolean onPlay): Initializes the BirdCard with respective values

Enum BonusCard

Overview: This enum is primarily for the sake of encapsulation so as to correspond with images and actions easier.

Attributes

- final String name: The name of the BonusCard

Methods

- BonusCard(String name)

Setup Algorithm

1. The main method in the Wingspan class starts the game on a newly created MenuPanel.
2. The MenuPanel contains a basic interface to select the number of players in the game and a play button. It runs the startGame method in the Wingspan class afterwards with the proper variables and a newly initialized GamePanel, setting the GamePanel as the panel to be displayed by the Wingspan class.
3. The startGame method creates the Game class, which instantiates new Player classes (that instantiate their own Board classes with grids of Spot instances), creates the global decks, and draws five cards; two bonus cards; and five food tokens for each player to select a total of five items and one bonus card. Others are discarded.
4. The Round Algorithm is run.
5. The endGame method of the Game class will run the bonusAction method for each card and add the bonus points to each player in the playerPoint array.
6. The leaderboard and a victor will be displayed by the GamePanel as well as an option to restart. Upon restarting, the Setup Algorithm runs again.

Round Algorithm

1. The Game class sets the first player's (or the next player from the previous round's) marker to true and displays their board on the GamePanel.
2. The Turn Algorithm is run until the nextRound method is called. The Game class increments the round attribute and runs the Scoring Algorithm.
    a. If the round attribute of the Game reaches 4, the endGame method is called in the Game class, and the Round Algorithm terminates.
    b. Otherwise, the Round Algorithm continues to loop.
3. Each Player's actionCubes attribute is initialized to one less than the round before.the GamePanel with the option to restart. Restarting will display the MenuPanel once again to restart the overall game loop.

Turn Algorithm

1. The GamePanel listens for a mouseClicked event and internally manages code relating to graphics (which may include basic variables not detailed in the UML) until an action is performed.
2. The Player's actionCubes will be decremented upon any action selected by a player, and the Board's actionCube will have its coordinate set to the respective spot.
    a. If the action is playing a card, the Card Placement Algorithm is run.
    b. Otherwise, the Action Algorithm is run.
3. After the action is completed, the nextTurn method of the Game class will be run, restarting the Turn Algorithm on the next player's turn. If the next Player's actionCubes count is equal to 0, the nextRound method is called, and the Turn Algorithm terminates.

Card Placement Algorithm

1. The playCard method is run in the respective Player class to determine if the card was legal to play.
2. If legal, the method plays it by calling the subsequent playCard method in its Board instance, returning true if successful and false otherwise.
3. If successful, the GamePanel will display the option to run the card's on-play effect, running the runAction method in the Player class if so.
4. The respective finishedActions count in the Board instance will be incremented and the actionCube will be set to [-1, -1].

Action Algorithm

1. The placeAction method will be run to launch a loop in the Board instance where the initial position of the actionCube, an unoccupied spot, will affect the player with its respective trait, be that adding eggs, gaining food, or drawing a bird card.

2. The GamePanel will give the option to trade if the spot on the board offers, running the respective trade.
3. The loop will continued with the moveAction method - the method to move the actionCube left - being called until the actionCube's first value is -1, at which point the actionCube will be set to [-1, -1] and the respective finishedActions count will be incremented.
    a. The GamePanel will give the option to activate each card it crosses over if possible. If activated, the runAction method will be run in the Player instance to activate the action of the BirdCard's actionId.

Scoring Algorithm

1. The Game class will evaluate the round attribute and provide a specific criteria on which to assign points.
2. A nested loop runs through each Spot of each Board of each Player instance and adds points to the playerPoints array based on the given criteria.